

ASSIGNMENT 2

Factory Method and Abstract Factory Design Patterns

Course: Software Design Patterns

Student: Sannder Samarin

Group: SE-2537

Astana, 2026

1. Introduction

The purpose of this assignment is to implement two creational design patterns in Java: Factory Method and Abstract Factory. The project is a small console-based logistics application. It allows the user to choose a delivery method and a user interface platform. The main idea is to separate object creation from the code that uses these objects.

The application demonstrates how design patterns can make code more flexible and easier to extend. For example, a new type of transport or a new user interface family can be added without changing the main application logic significantly.

2. Project Requirements and Technologies

Item	Description
Programming language	Java
Main patterns	Factory Method and Abstract Factory
Application type	Console application
Development environment	IntelliJ IDEA or another Java IDE
External libraries	Not required

3. Factory Method Pattern

The Factory Method pattern defines a method for creating an object, but the decision about which concrete object to create is delegated to subclasses. In this project, the common product is represented by the Transport interface. The concrete products are Truck and Ship.

The abstract creator is the Logistics class. It declares the createTransport() method and contains the shared planDelivery() method. The subclasses RoadLogistics and SeaLogistics decide which transport object should be created.

Main classes involved:

- Transport - common interface for all delivery vehicles.
- Truck - implements delivery by road.
- Ship - implements delivery by sea.
- Logistics - abstract creator with the common delivery process.
- RoadLogistics - creates a Truck object.
- SeaLogistics - creates a Ship object.

Example of the pattern in the project:

```
public abstract class Logistics {  
    public abstract Transport createTransport();  
  
    public void planDelivery(String cargo, String destination) {  
        System.out.println("Planning delivery...");  
        Transport transport = createTransport();  
        transport.deliver(cargo, destination);  
    }  
}
```

4. Abstract Factory Pattern

The Abstract Factory pattern provides an interface for creating families of related objects. In this project, the related objects are a button and a checkbox. Each platform has its own family of components, so Windows components are created together and macOS components are created together.

Main classes involved:

- Button and Checkbox - abstract product interfaces.
- WindowsButton and WindowsCheckbox - Windows product family.
- MacOSButton and MacOSCheckbox - macOS product family.
- GUIFactory - abstract factory interface.
- WindowsFactory - creates Windows components.
- MacOSFactory - creates macOS components.
- DeliveryApplication - uses the factory through interfaces.

```
public interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}
```

5. Application Logic

The program starts in Main.java. The user enters two values: the delivery mode and the UI platform. The first value selects either RoadLogistics or SeaLogistics. The second value selects either WindowsFactory or MacOSFactory.

After the objects are selected, they are passed to DeliveryApplication. This class creates a button and a checkbox using the selected GUI factory. It then calls the logistics object to plan and perform the delivery. The application works with interfaces, so it does not need to know the exact concrete classes during the main workflow.

6. Project Structure

```
src/main/java/  
  logistics/  
    Transport.java  
    Truck.java  
    Ship.java  
    Logistics.java  
    RoadLogistics.java  
    SeaLogistics.java  
  ui/  
    Button.java  
    Checkbox.java  
    WindowsButton.java  
    WindowsCheckbox.java  
    MacOSButton.java  
    MacOSCheckbox.java  
    GUIFactory.java  
    WindowsFactory.java  
    MacOSFactory.java  
  application/  
    DeliveryApplication.java  
    Main.java
```

7. Example of Program Execution

One possible execution is shown below. In this example, the user selects sea delivery and the macOS interface family.

```
Enter delivery mode (ROAD or SEA): SEA
Enter UI platform (WINDOWS or MACOS): MACOS

Delivery mode: SEA
UI platform: MACOS
Rendering macOS button
Rendering macOS checkbox
Planning delivery...
Ship delivers laboratory equipment to Aktau warehouse by sea.
```

The program also checks the input values. If the user enters an unsupported delivery mode or platform, the program prints an error message and stops instead of creating an invalid object.

8. UML Diagrams

The UML diagrams are provided separately in the UML.md file in the repository. The first diagram describes the Factory Method structure, including the Logistics creator and the Transport products. The second diagram describes the Abstract Factory structure, including the GUIFactory and the two platform-specific product families.

9. Conclusion

During this assignment, I implemented two creational design patterns in Java. The Factory Method pattern was used to select and create a transport object depending on the delivery type. The Abstract Factory pattern was used to create compatible UI components for different platforms.

Using these patterns makes the application more organized because object creation is separated from the main business logic. It is also easier to add new delivery methods or new interface platforms later. This assignment helped me understand how interfaces, inheritance, and polymorphism can be combined with design patterns in a practical project.

10. Repository

GitHub repository: https://github.com/sannder01/ass2_sdp